Explicit Adaptive Runge - Kutta Method

- Task 1.1

It takes a single explicit Runge Kutta step of size h, from the point uold to produce the next approximation, unew.

```
function unew = RK4step(f, told, uold, h)

    % RK4, with formulas given in the project
    k1 = f(told, uold);
    k2 = f(told + h/2, uold + h/2 * k1);
    k3 = f(told + h/2, uold + h/2 * k2);
    k4 = f(told + h, uold + h * k3);

    % update step for RK4
    unew = uold + h/6 * (k1 + 2*k2 + 2*k3 + k4); %y_n+1
end
```

Runge - Kutta

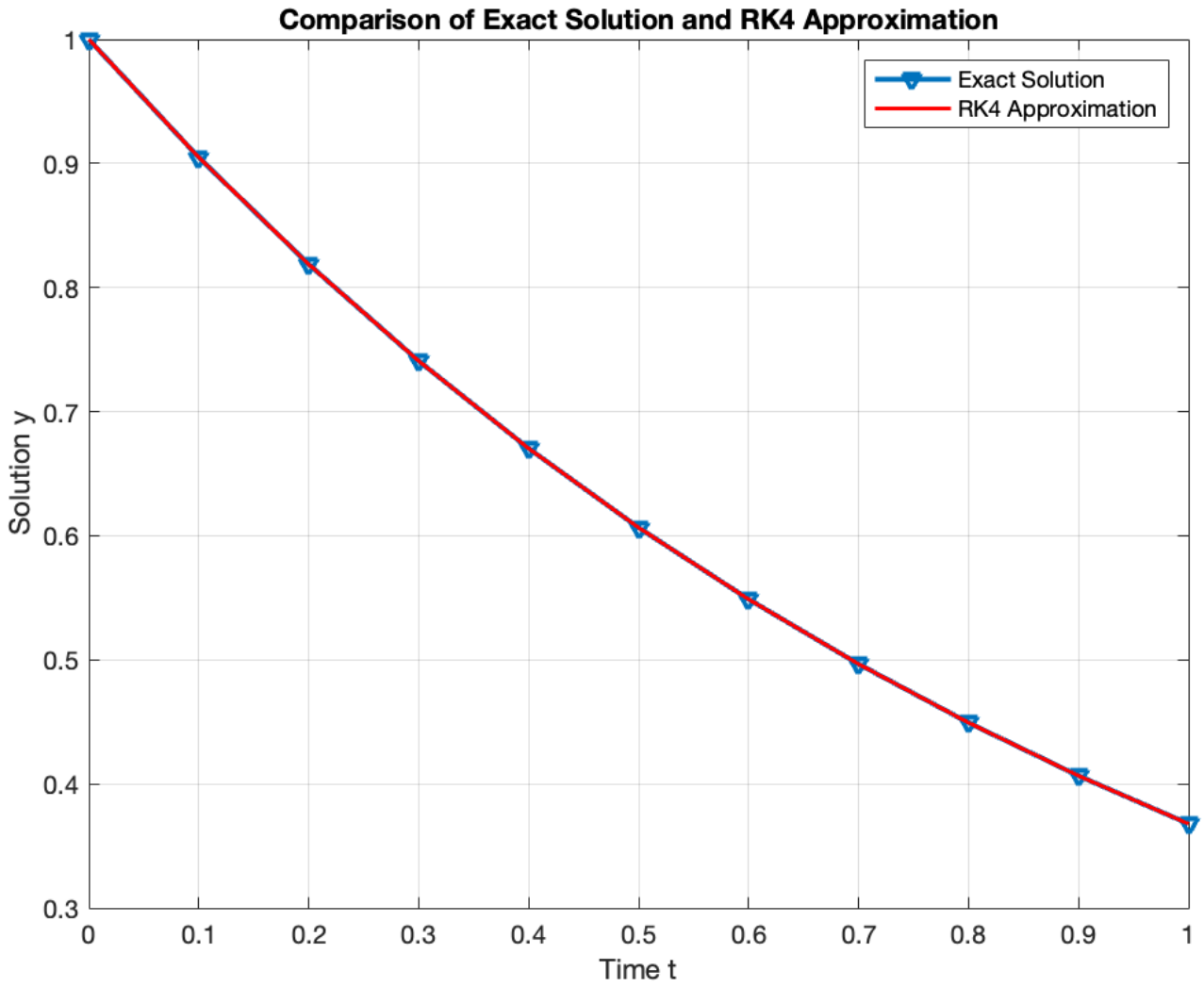
2. Using RK4step we construct a function.

```
function [tgrid, approx_values, endpoint_error] = RK4int(f, y0, t0, tf, N)

    % step size
    h = (tf - t0) / N;
```

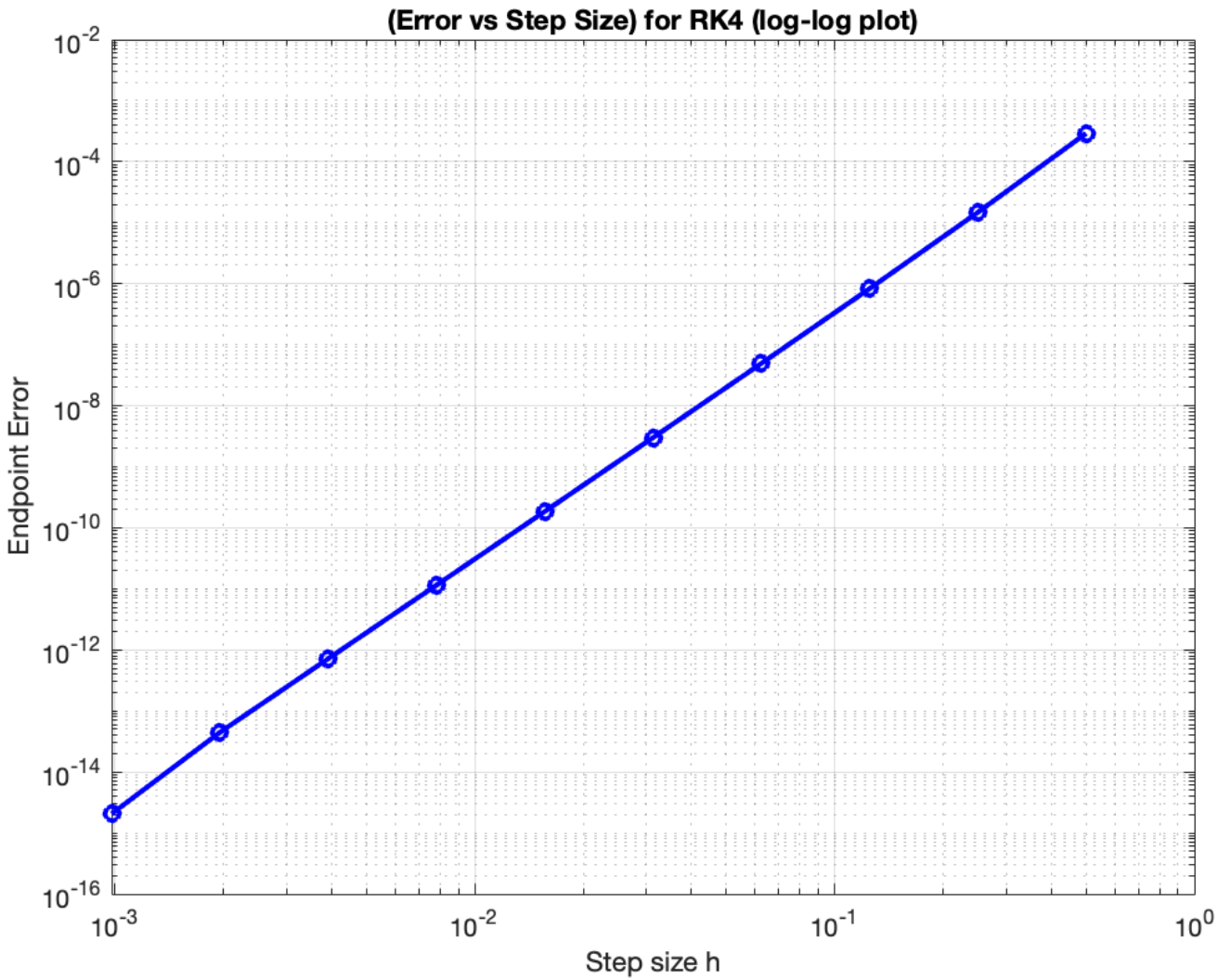
It approximates the solution using N steps on the interval $[t_0, t_f]$. In this function we find the exact solution and endpoint error.
Note: We need the matrix exponential e^{tA} to compute the exact solution and the error, using `expm(A)`.

Plot


Comparison of Exact Solution and RK4 Approximation
Exact Solution
RK4 Approximation
Solution y
Time t
1
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1

Runge-Kutta method

```
function errorVShRK4(f, y0, t0, tf)
```



Thoughts on the plot

The global error depends on the step size of h, or on the the total number of steps N taken to reach the end point. We use log log as we want to see how the error depends on the step size h. We have a straight line with a slope of 4, meaning for every 4 units of change in y, there is one unit of change in x. The global error is $O(h^4)$.

Runge - Kutta Method

- Task 1.2 Compute a local error estimate

```
function [unew, err] = RK34step(f, told, uold, h)

    Y1 = f(told, uold);
    Y2 = f(told + h/2, uold + h/2 * Y1);
    Y3 = f(told + h/2, uold + h/2 * Y2);
    Z3 = f(told + h, uold - h * Y1 + 2 * h * Y2);
    Y4 = f(told + h, uold + h * Y3);

    % update
    unew = uold + h/6 * (Y1 + 2*Y2 + 2*Y3 + Y4);

    % RK3 estimate using Z3
    u_RK3 = uold + h/6 * (Y1 + 4*Y2 + Z3);

    % Error estimate
    err = norm(u_RK3 - unew); %zn+1-yn+1
end
```

Runge - Kutta Method

- Task 1.3 Tolerance

```
function hnew = newstep(tol, err, errold, hold, k)


    hnew = (tol / err)^(2 / (3 * k)) * (tol / errold)^(-1 / (3 * k)) * hold;
end
```

Runge - Kutta Method

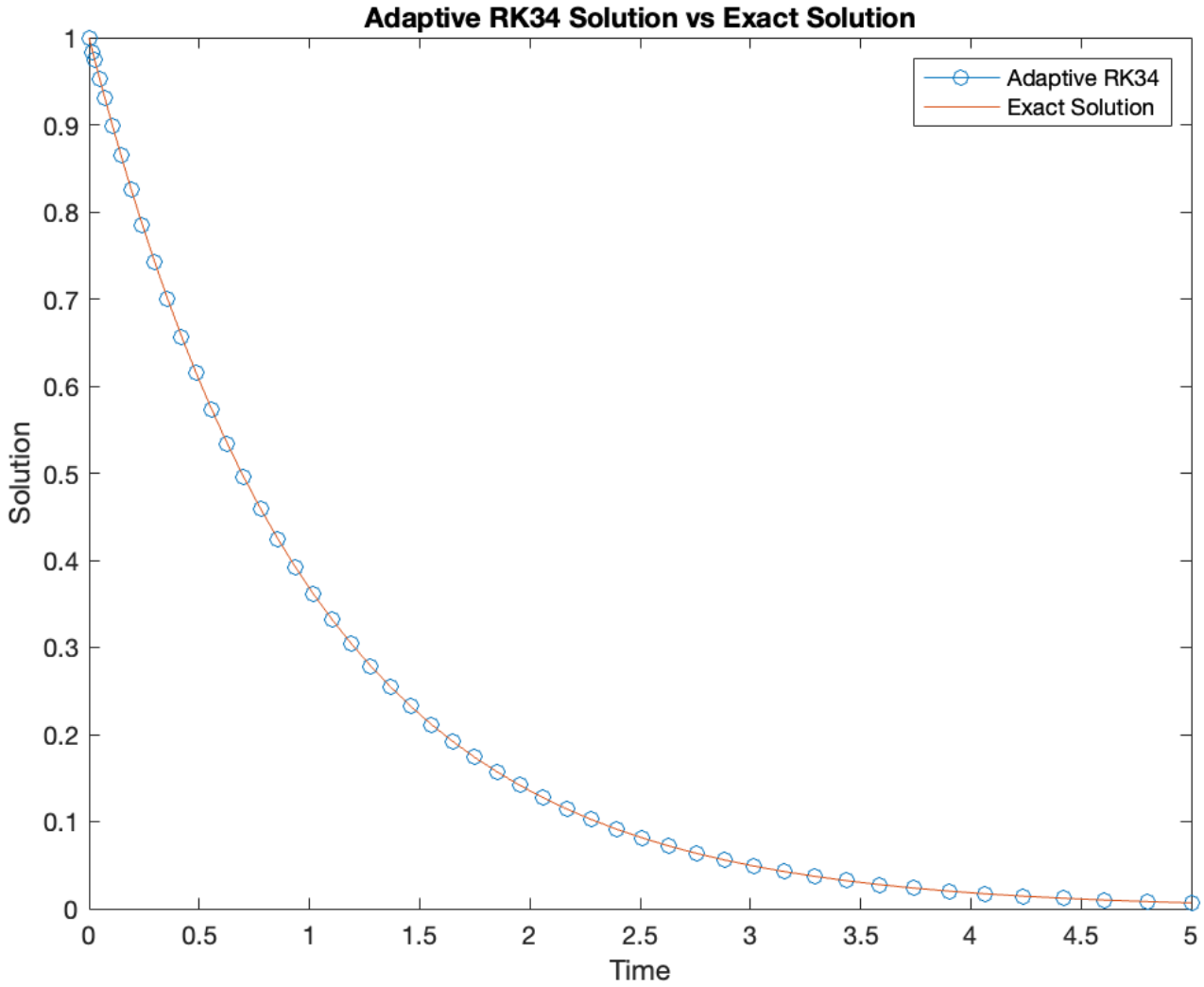
- Task 1.4 Combining RK34 and newstep

```
function [t, y] = adaptiveRK34(f, t0, tf, y0, tol)
    t = t0;
    y = y0;
    t_values = t;
    y_values = y0';

    % initial step size based on the given formula
    h = abs(tf - t0) * tol^(1/4) / (100 * (1 + norm(f(t0, y0))));
    k = 4;  % order of the error estimator for RK34 said on the project
```

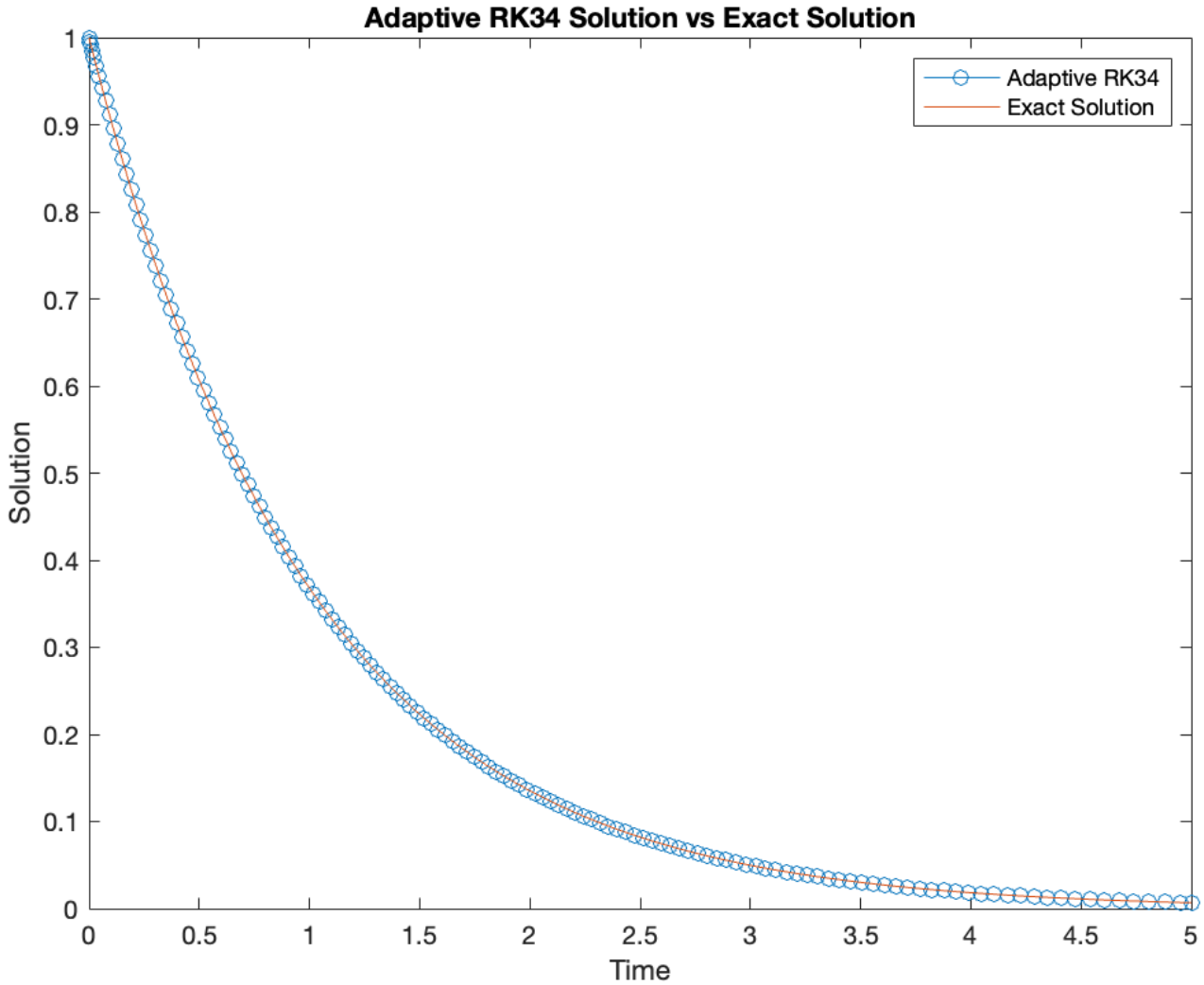
Plot

$tol = 10^{-6}$; $t_f = 5$; Number of steps $= 51$; Final solution: 0.00671

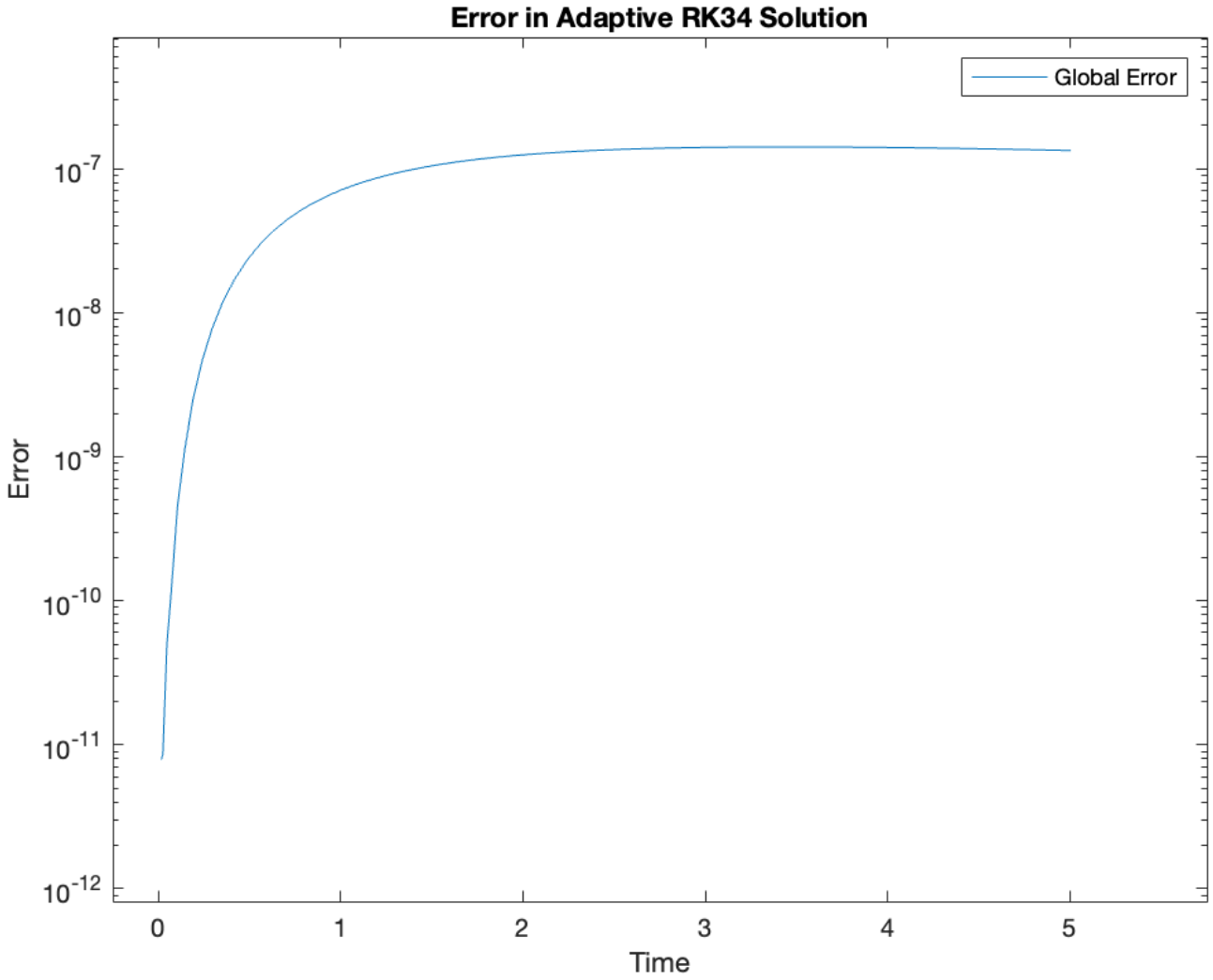


Plot

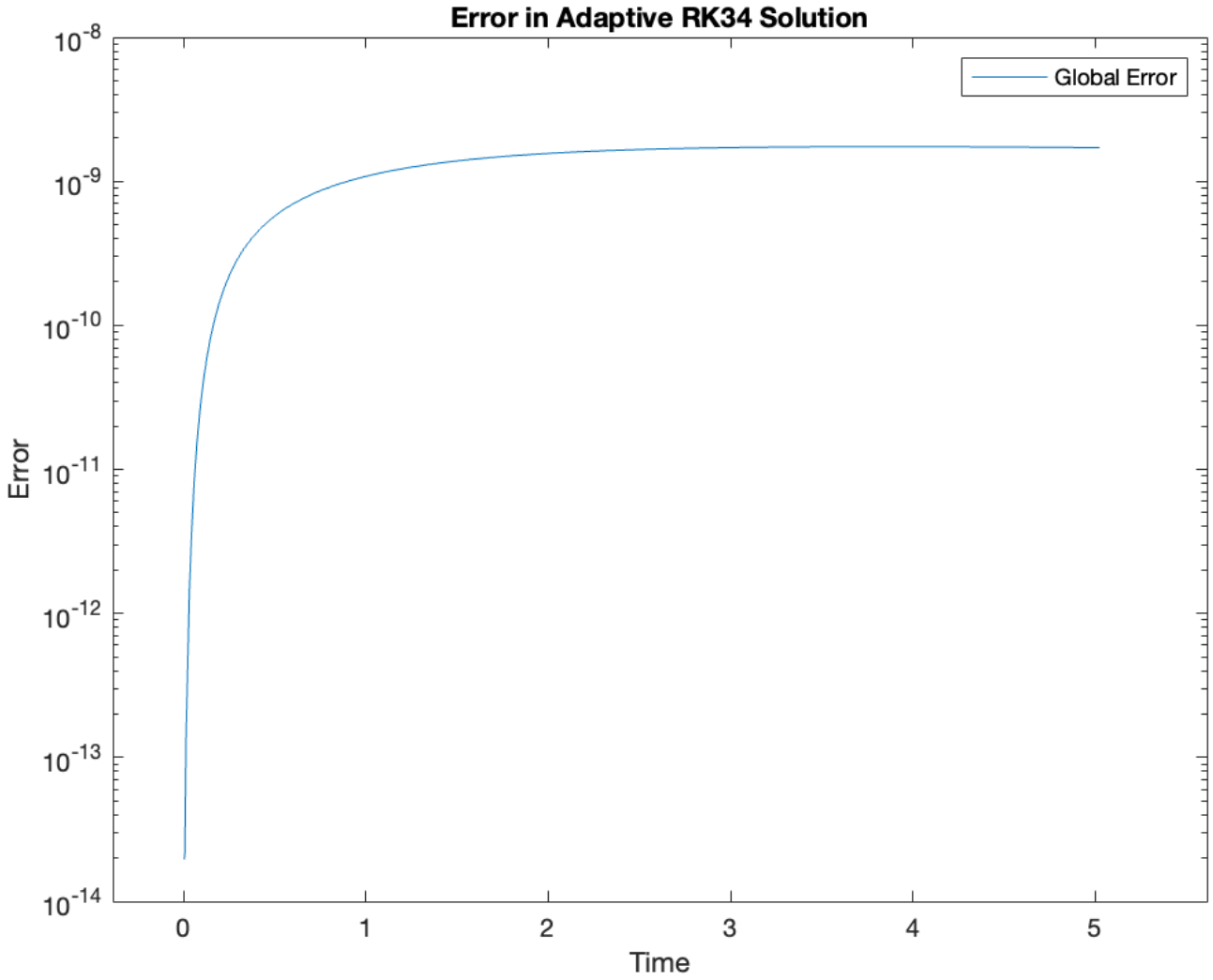
$tol = 10^{-8}$; $t_f = 5$; Number of steps = 140; Final solution: 0.00658
Note: The final time taken by MATLAB was 5.02.



Global Error for $tol = 10^{-6}$



Global Error for $tol = 10^{-8}$



Runge - Kutta Method

The value of e^{-5} is 0.0067, showing that the solution is actually pretty close for tolerance 10^{-6} and very near for 10^{-8}. When we have 10^{-8} we have an increment of steps (140) and a less accurate solution because the final time is 5.02 instead of 5. When we have 10^{-6} the number of steps is 51 (larger steps) and more accurate solution because the final time reached by matlab is 5.

- Note: If we add a controller to adjust the step size, we have a better solution, reaching 0.00674 in both cases, because in this case they final time is 5.

2. A nonstiff Problem - Lotka - Volterra equations

Task 2.1 Lotka Volterra function

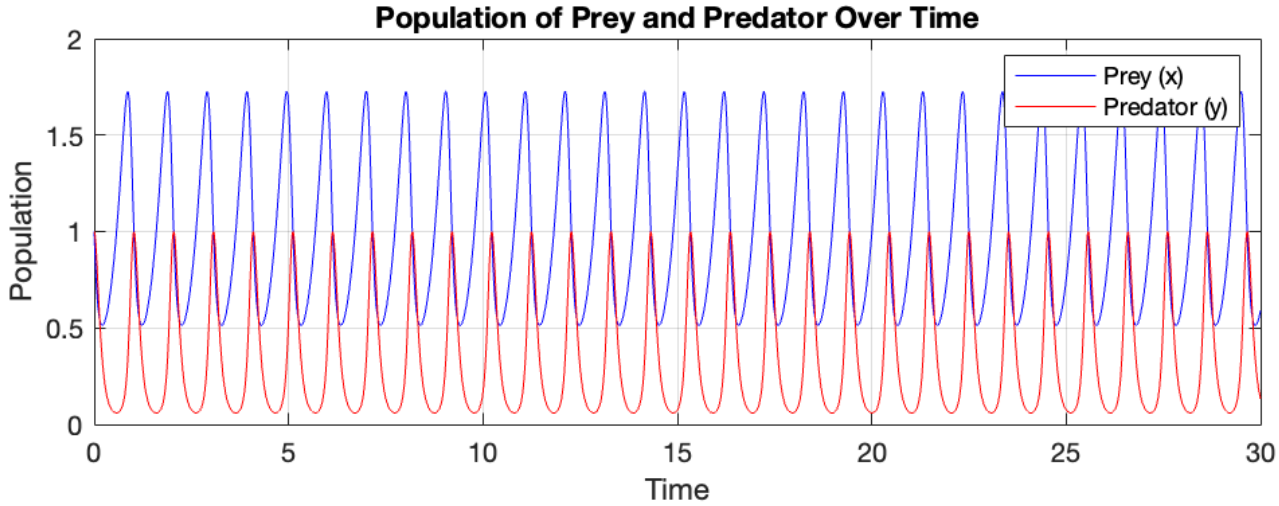
- $x = prey = rabbits = u(1)$
- $y = predator = foxes = u(2)$
- $xy =$ probability of encounter

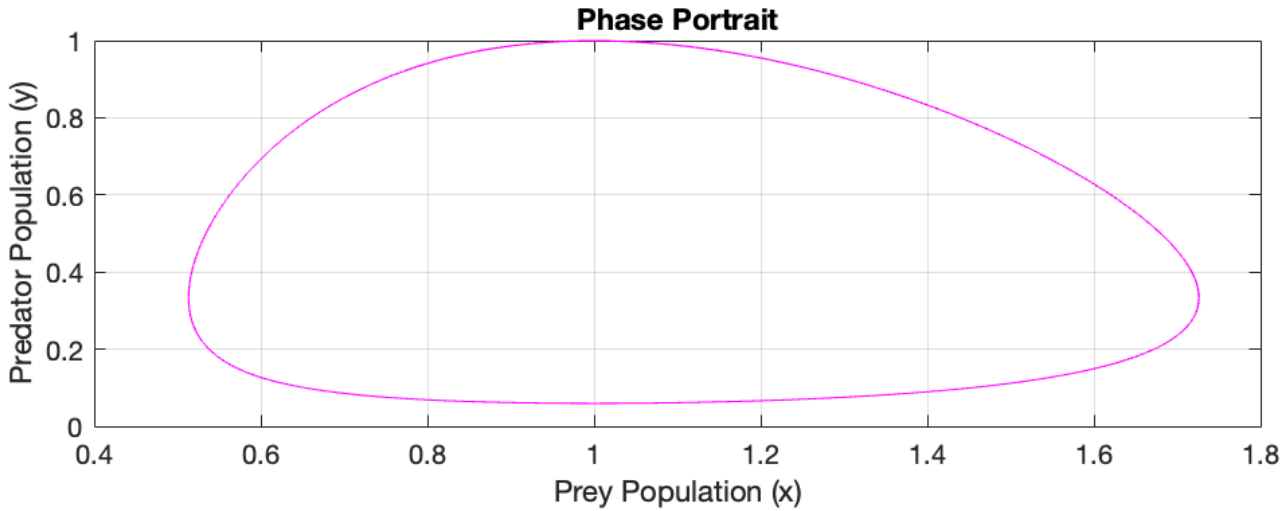
```
function dudt = lotka(t, u)
    % given values
    a = 3;
    b = 9;
    c = 15;
    d = 15;

    % System of ODEs as given in the hint
    dudt = [a * u(1) - b * u(1) * u(2); c * u(1) * u(2) - d * u(2)];
end
```

Lotka-Volterra

Using $tol = 10^{-6}$, with 30 full periods, with initial conditions (1,1)



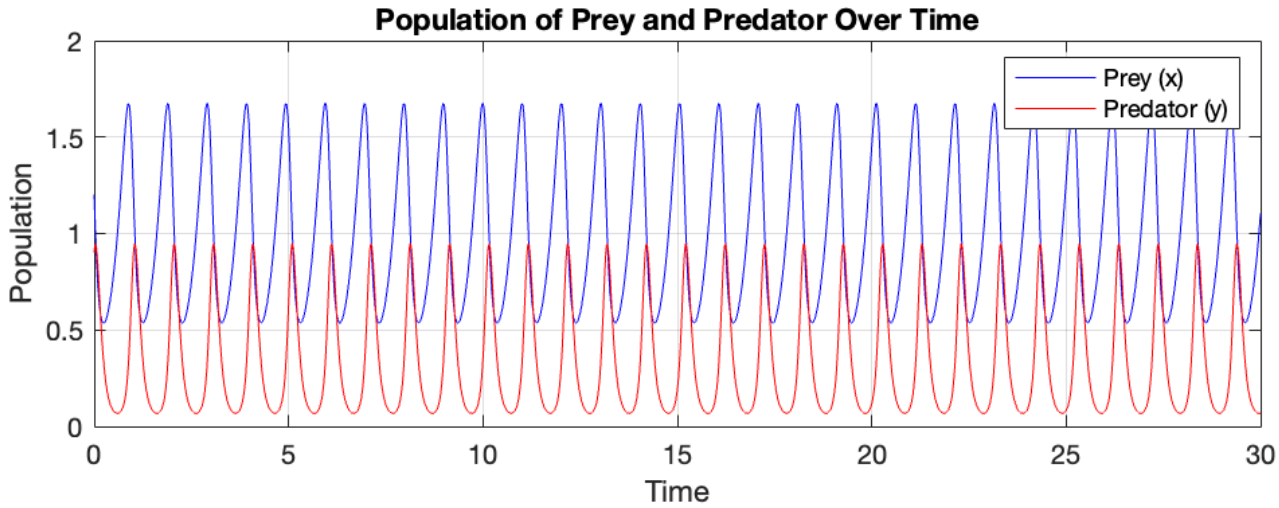


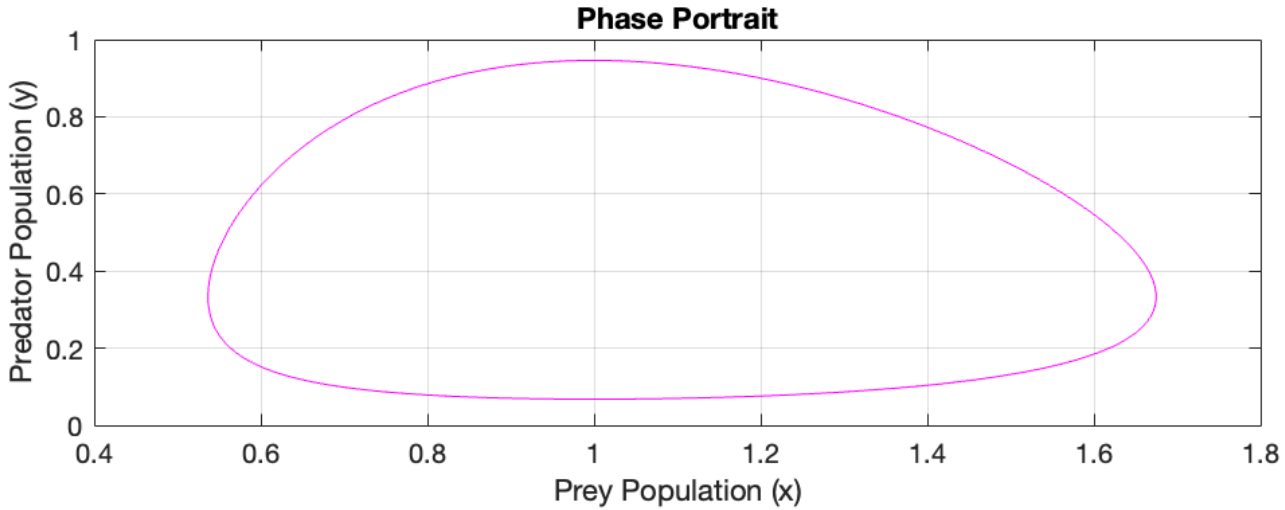
Thoughts on the plot

The system is periodic, the oscillations goes from around 0 to more than 1.5. We can see that we have a closed orbit in the phase portrait and a recurring patterns in the time series plot. It seems we have a period of around 1. The graph looks very similar with tolerance 10^{-8}.

Lotka-Volterra

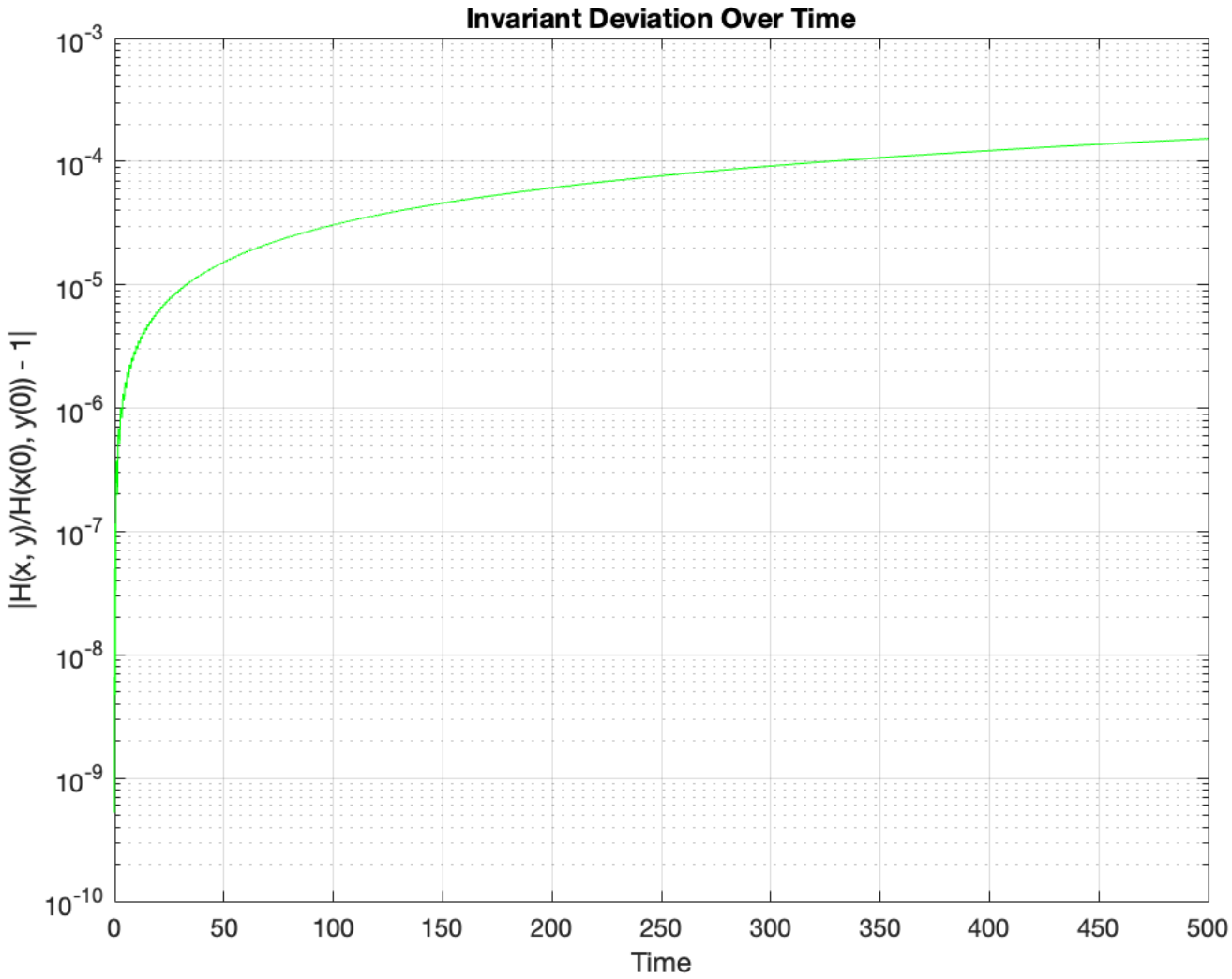
Using $tol = 10^{-6}$, with 30 full periods, with initial conditions (1.2,0.9). Period of around 1.





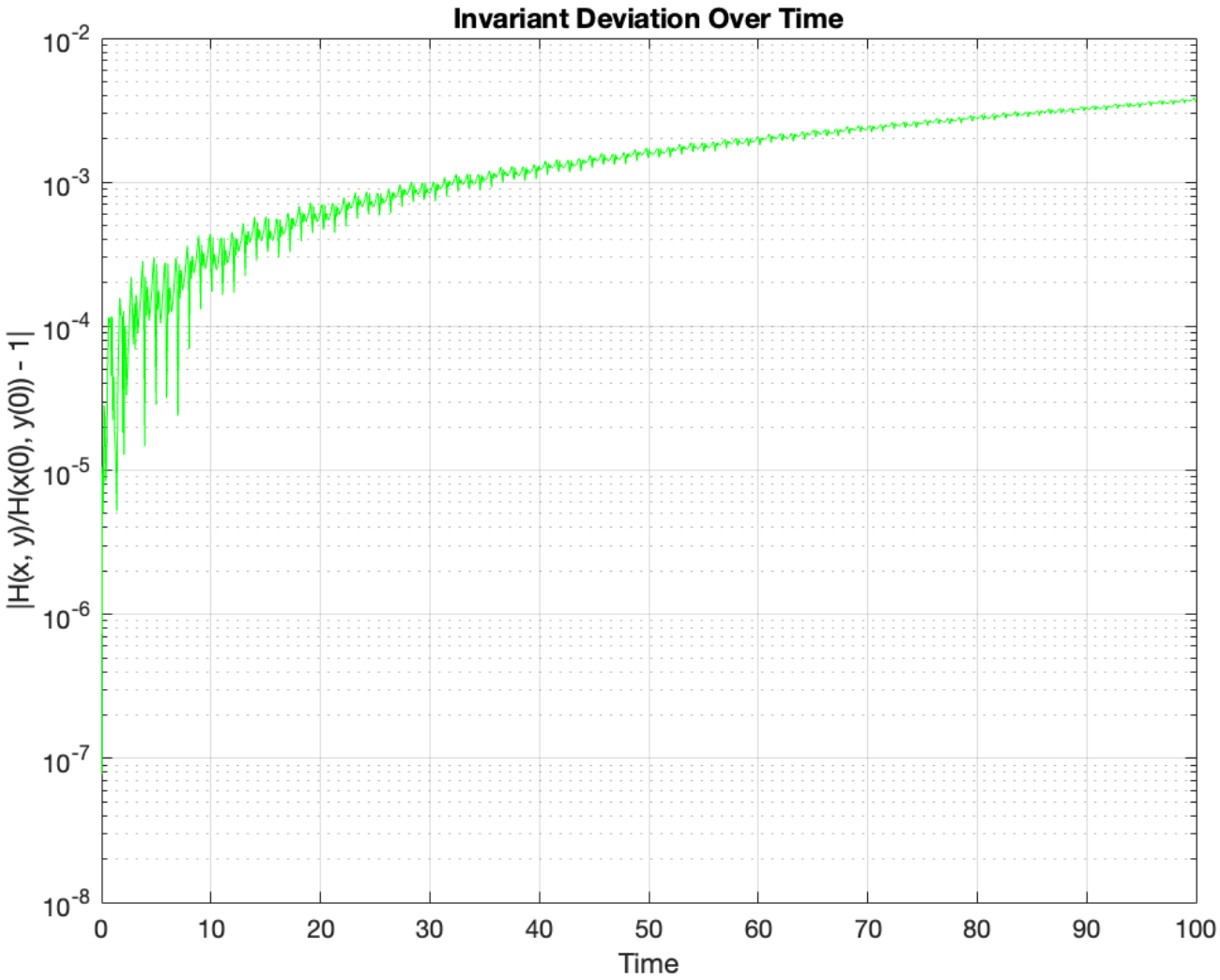
Lotka-Volterra

Using $tol = 10^{-6}$, with 500 full periods, with initial conditions $(1, 1)$.



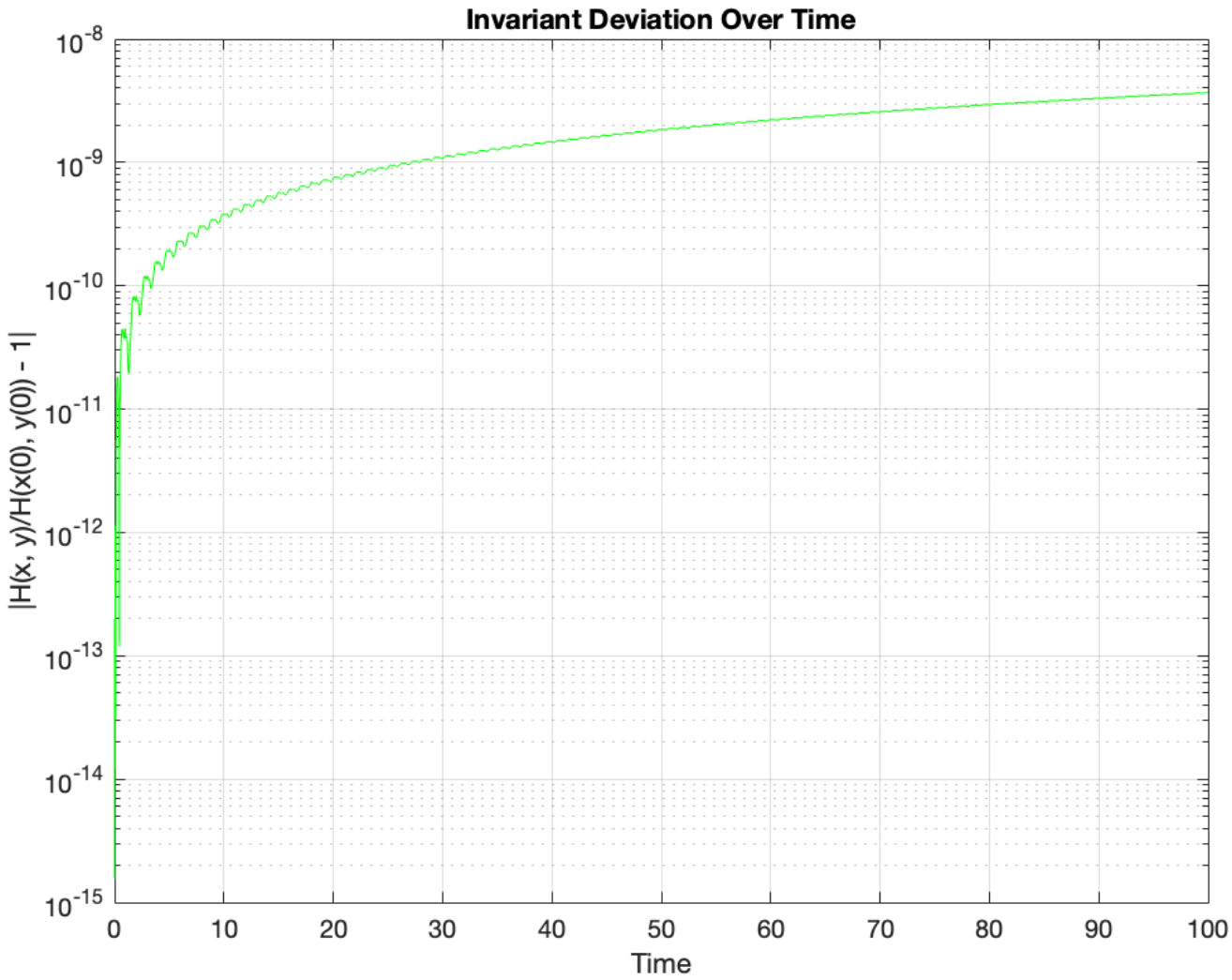
Lotka-Volterra

Using $tol = 10^{-3}$, with 100 full periods, with initial conditions $(1, 1)$.



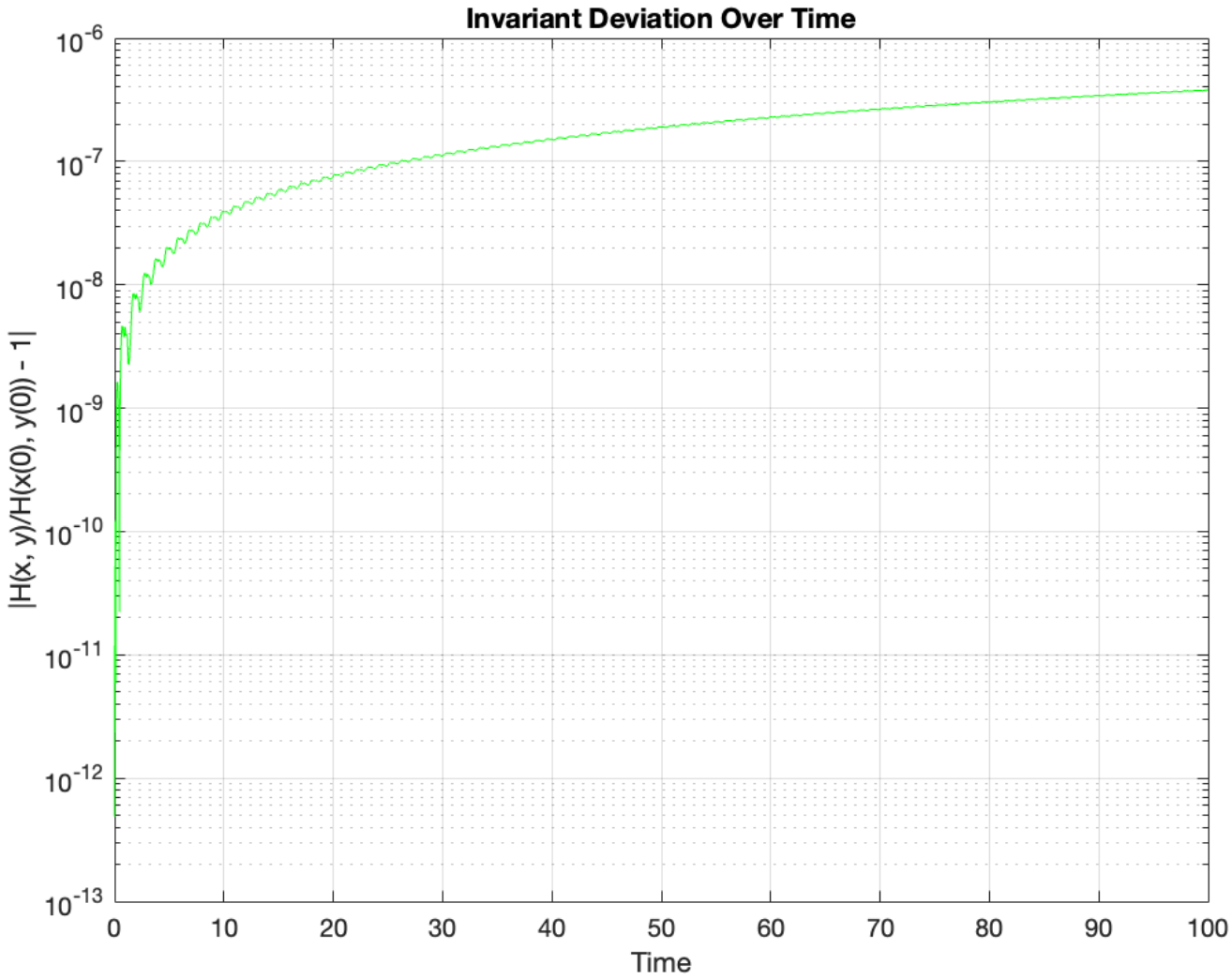
Lotka-Volterra

Using $tol = 10^{-6}$, with 100 full periods, with initial conditions $(1, 1)$.



Lotka-Volterra

Using $tol = 10^{-8}$, with 100 full periods, with initial conditions $(1, 1)$.



Thoughts on the plot

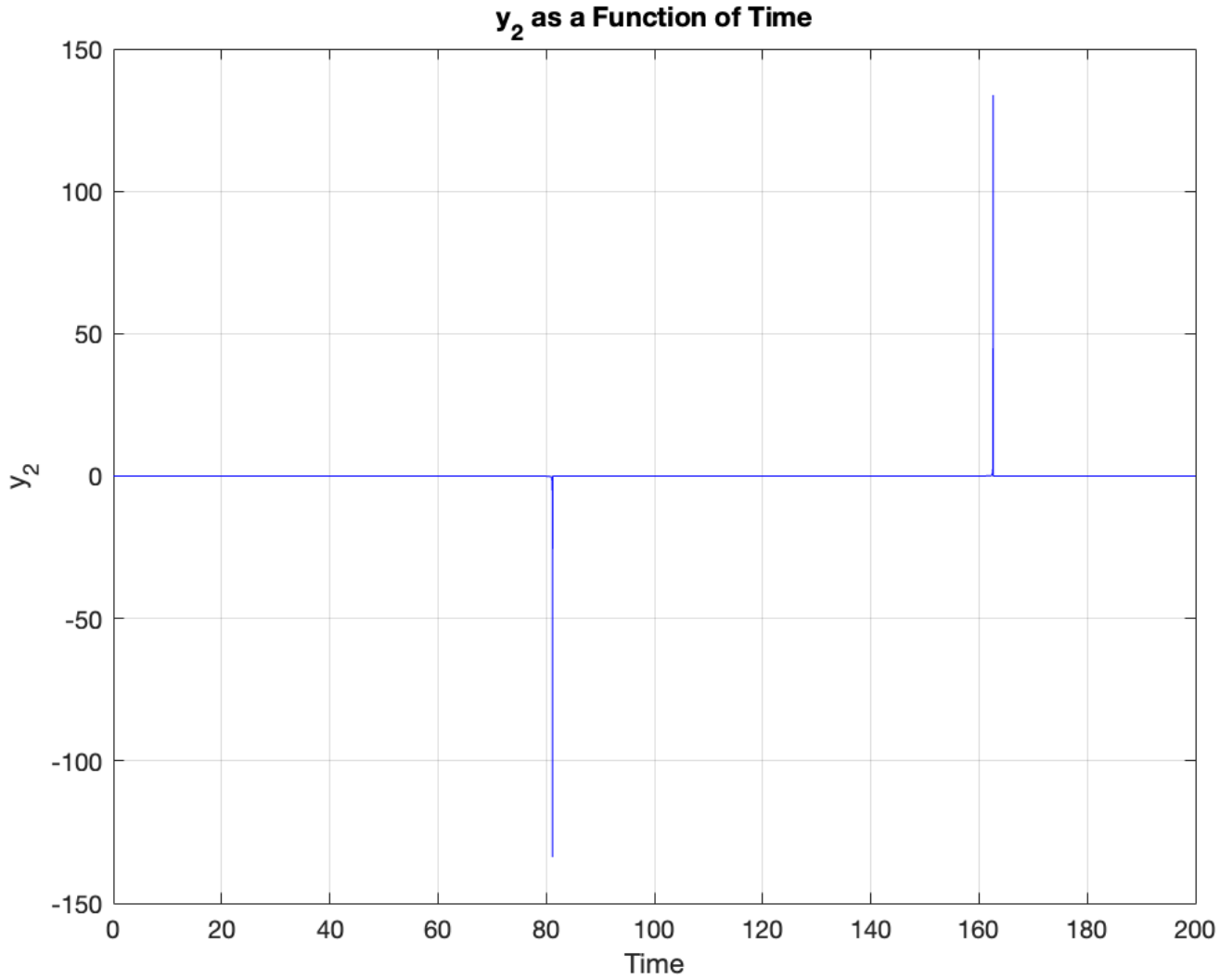
H(x,y) plots. The plot in theory would be constant, tending to zero it would be flat. But in practice we have numerical errors. We used lin-log, as we want to see how H changes over time using linear scale for x but with y we use a log scale as we want to see the small deviations. We can see that it does not stay near to zero, meaning it drifts away because of numerical errors.
Using an higher $tol = 10^{-8}$ we will have a high accuracy.

3. Nonstiff and stiff problems

- Van Der Pol equation

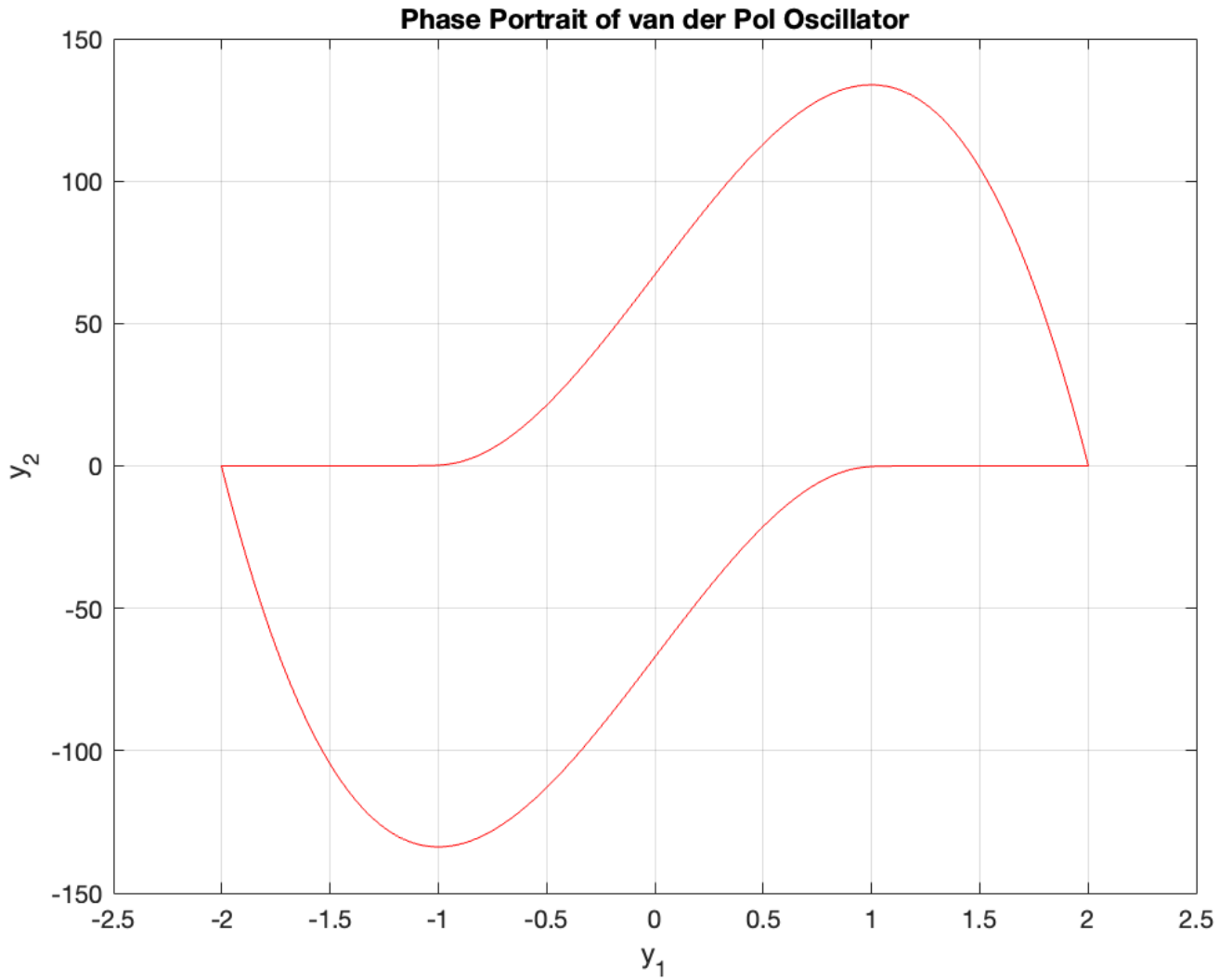
```
function dydt = vanderpol(t, y)
    mu = 100;
    dydt = [y(2); mu * (1 - y(1)^2) * y(2) - y(1)];
end
```

$y_0 = [2; 0]$; $tol = 10^{-6}$; solution component as function of time.

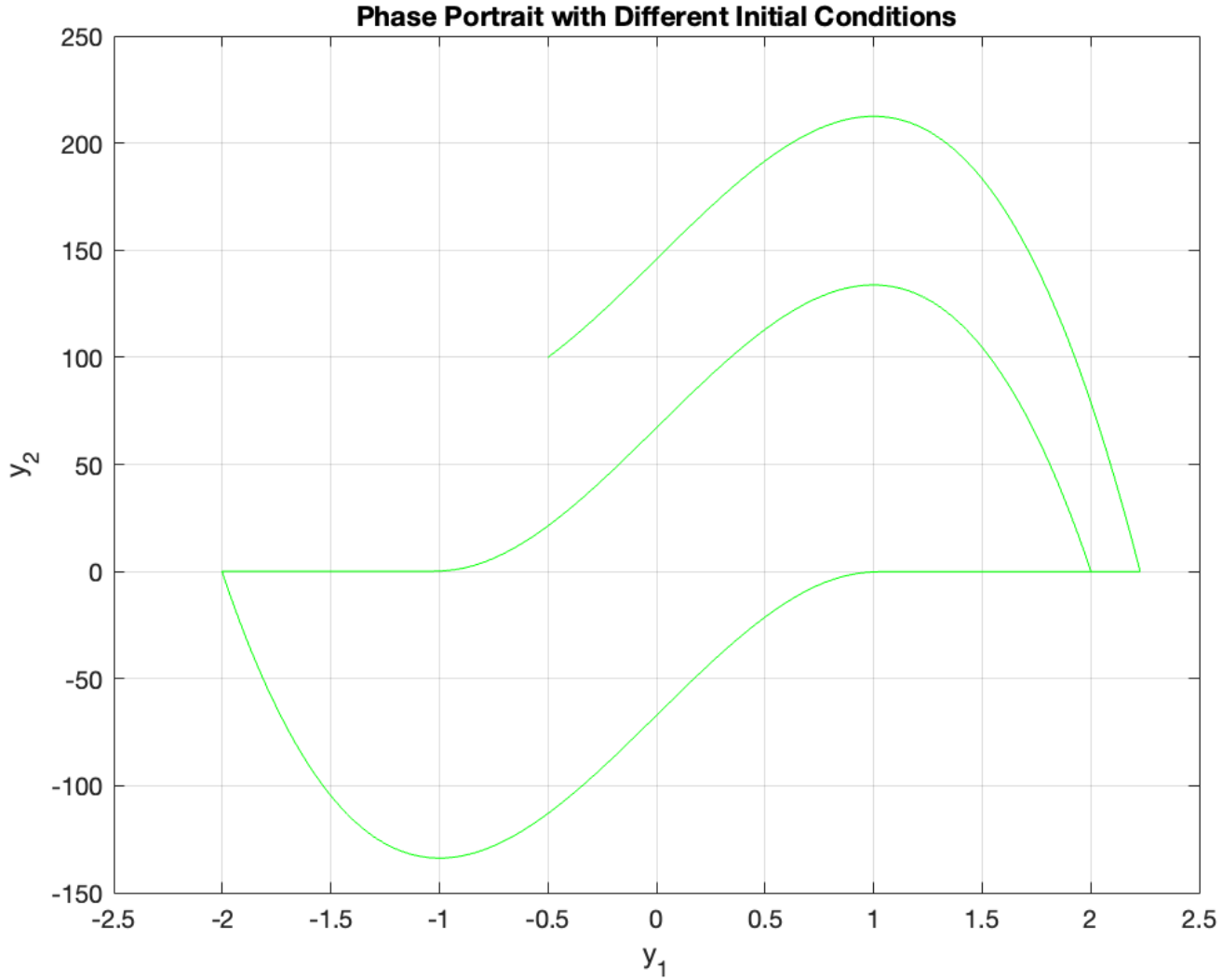


3.1

$y_0 = [2; 0]$; $tol = 10^{-6}$; y_2 as function of y_1 (phase portrait)



$y_0 = [0.5; 100]$; $tol = 10^{-6}$; y_2 as function of y_1 (phase portrait)

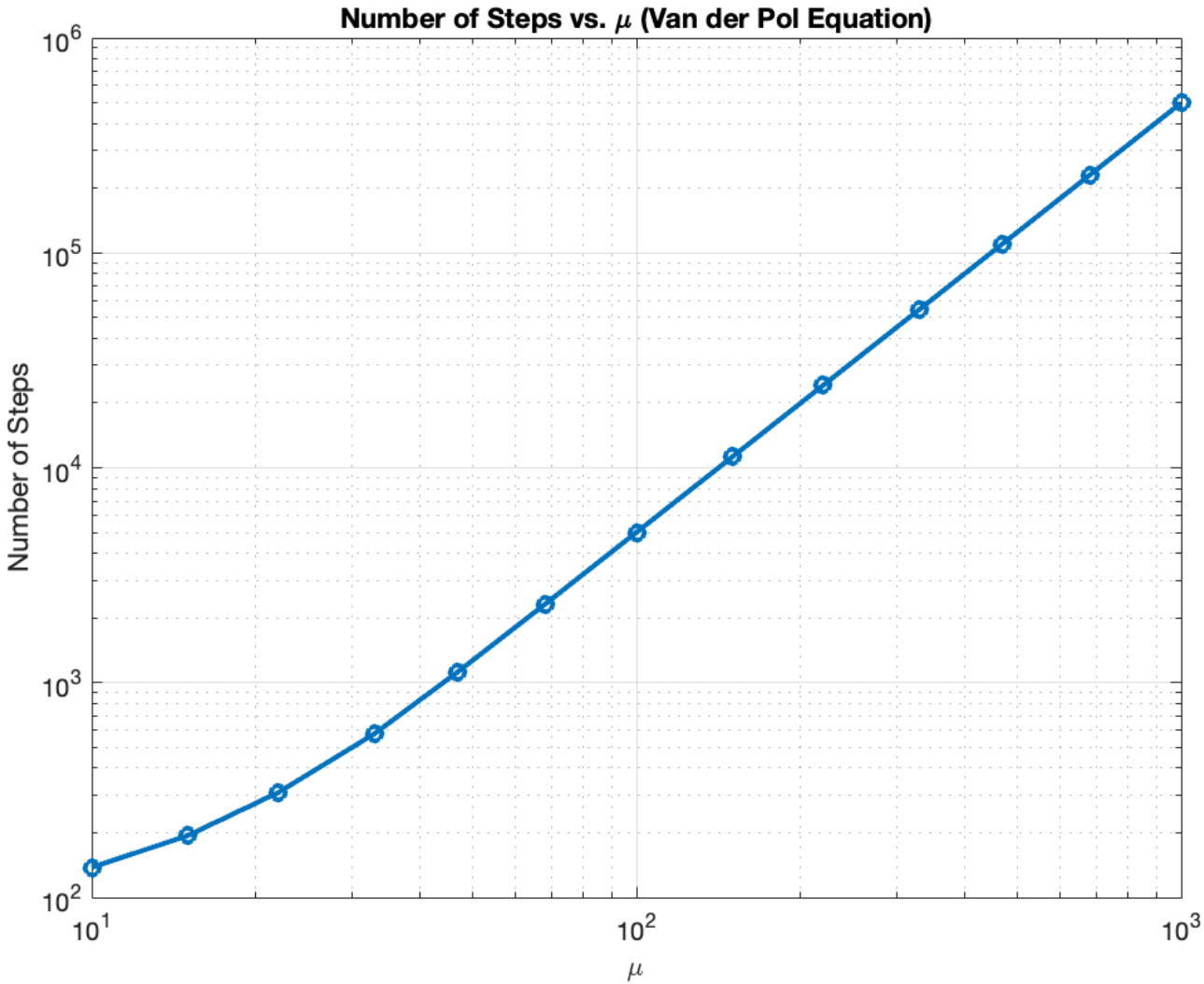


Thoughts on the plot

- Sharp peak means rapid change, we have pick of energy followed by slow / flat
- The phase portrait is a mix of oval and rectangular due to the rapid change
- We have a closed loop indicating a stable and periodic solution
- In the second plot we put a drastic value for y_2, and we see another trajectory that last for a short time and moves away from the limit cycle due this drastic value that we put, it does the same curve as the limit cycle and settle down, probably because it reaches a stable oscillations.

3.2

$\mu = [10, 15, 22, 33, 47, 68, 100, 150, 220, 330, 470, 680, 1000]$



Thoughts about the plot

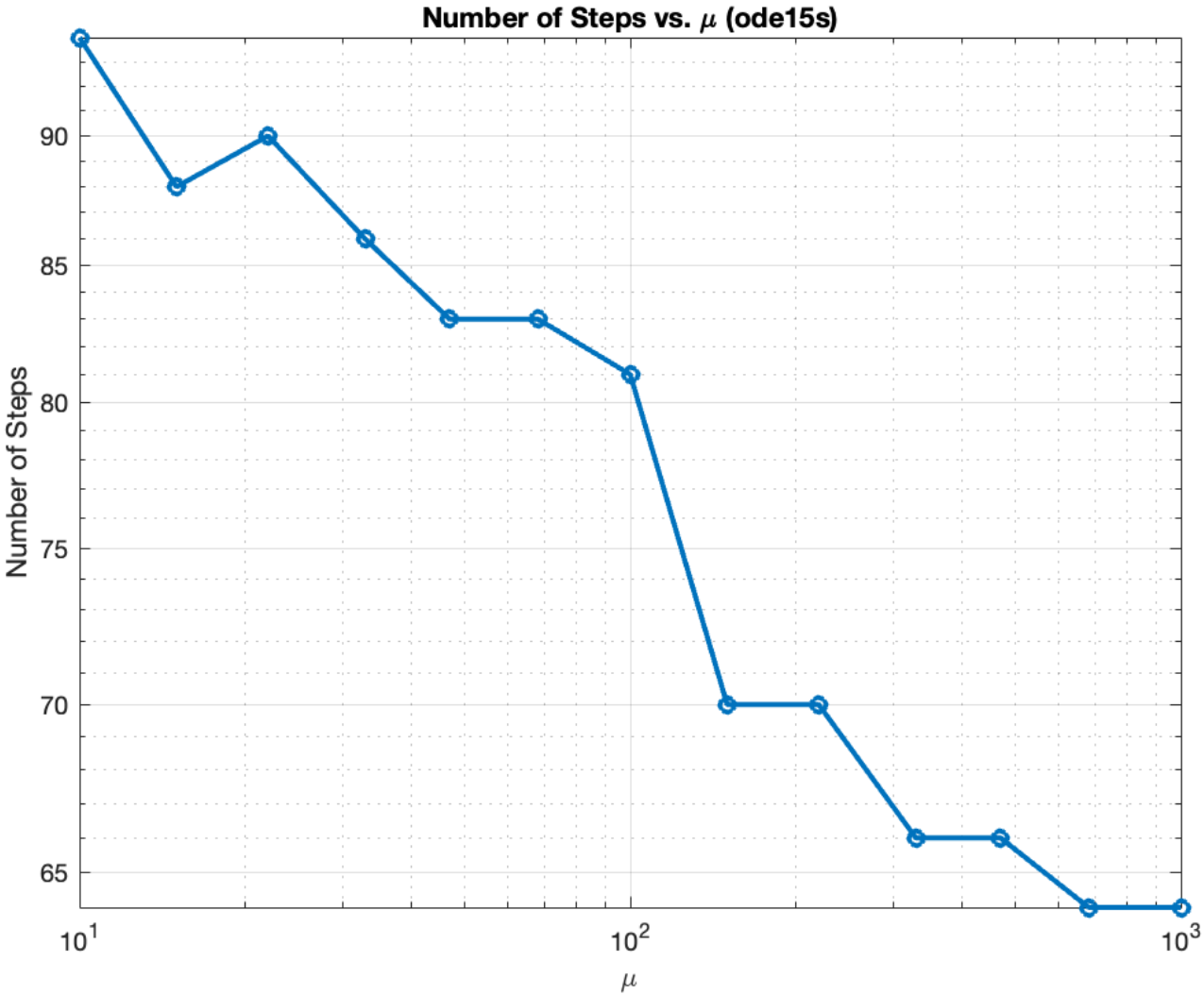
The slope is almost a straight line, we have a slope of $\frac{d-c}{b-a} = \frac{10^5-10^3}{10^2-10^1} = \frac{10^2}{10^1}$, meaning 2 units of change in y and 1 units of change in x. The stiffness depends on μ, and it increases if we increase μ, as it requires more computations the numbers of steps will grow, since in this case we have something that changes very quickly and something that change slowly.

3.3 odes

ode15s matlab solver to solve differential equations, using tol $= 10^{-6}$

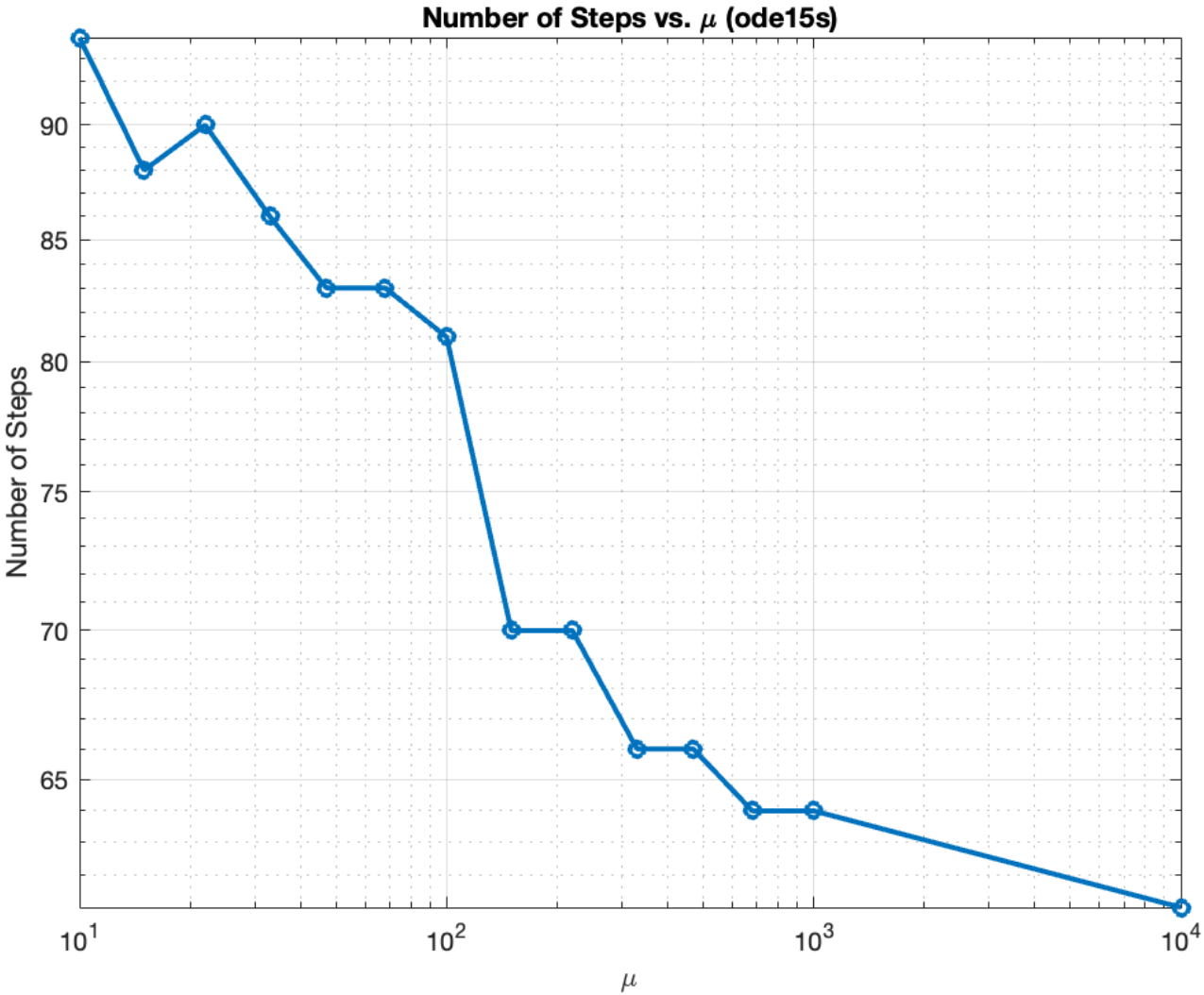
```
% ode15s solver
sol = ode15s(@(t, y) [y(2); mu * (1 - y(1)^2) * y(2) - y(1)], [t0 tf], initial_condition, options);
```

3.3

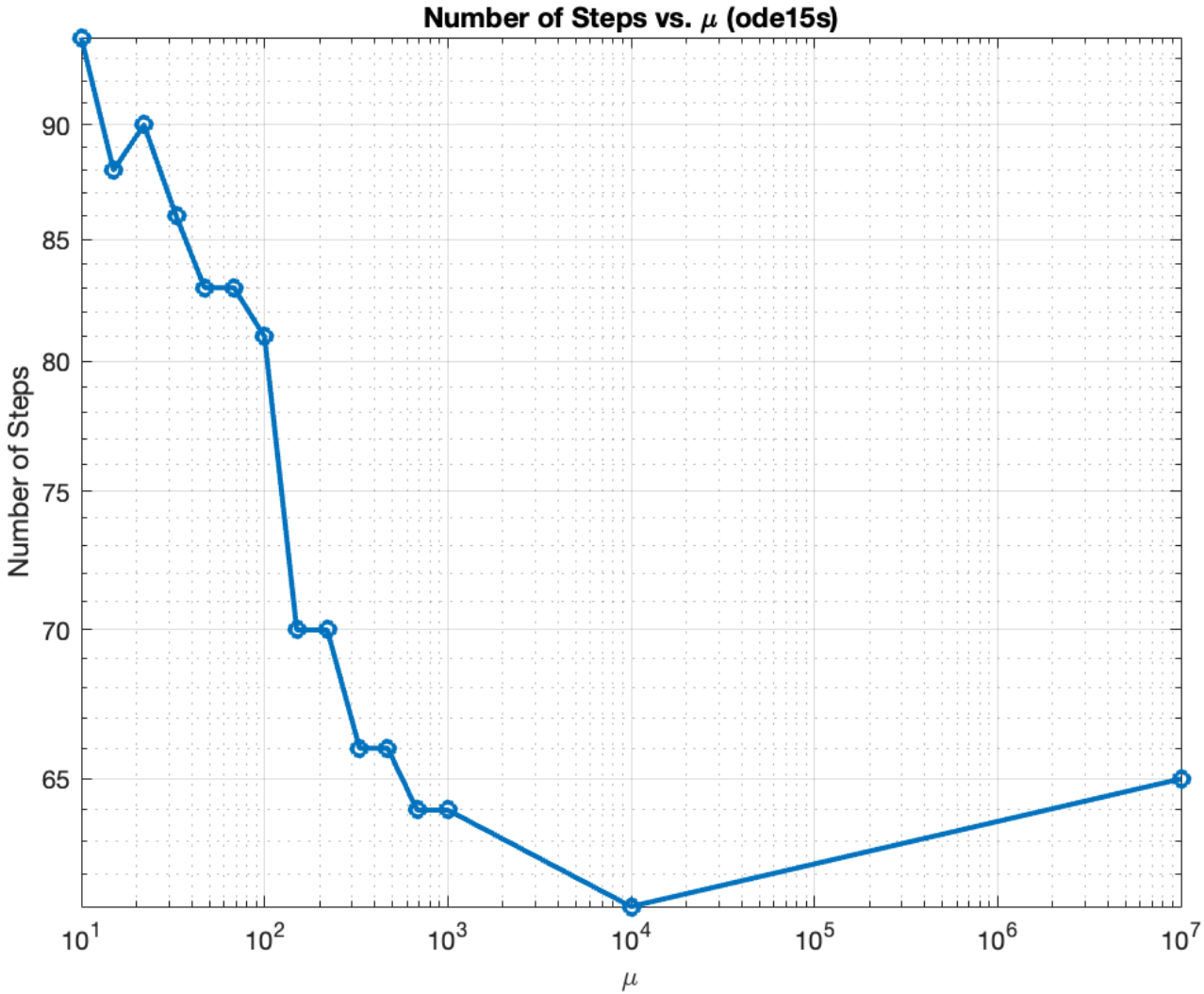

Number of Steps vs. μ (ode15s)
Number of Steps
90
85
80
75
70
65
10^1
10^2
10^3
μ

3.3

$\mu = 10000$


Number of Steps vs. μ (ode15s)
Number of Steps
90
85
80
75
70
65
10^1
10^2
10^3
10^4
μ

μ higher, $\mu = 10000000$



Thoughts on the solver

- It takes a significant less amount of steps, arriving at max 66 with respect to our solver, also for lager value of μ.
- This leads us to the conclusion the ode15s is more efficient
- Difference in the graphs: With RK34 solver the curve is steep, while with ode15s the curve is constant, if we put a limit it seems that the slope is tending to zero, because it's almost becoming straight. The scale used on the y axis is just 100 while the scale on the x axis is 10000000. The next slide will show a figure on how the curve with ode15s tends to zero.

lim

with `ylim([0 , 1000])`